

ALL SORT

Sorting Algorithms Practical Record

Bubble Sort

Program:

```
import time

# Function for Bubble Sort
def bubble_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        swapped = False

        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap adjacent elements
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # Stop if the array is already sorted
        if not swapped:
            break

# User Input
n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start timer
start_time = time.perf_counter()

# Perform Bubble Sort
bubble_sort(arr)

# End timer
end_time = time.perf_counter()

# Calculate execution time
execution_time = end_time - start_time

# Display results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case :  $O(n)$ ")
print("Average Case :  $O(n^2)$ ")
print("Worst Case :  $O(n^2)$ ")

print("\nSpace Complexity:  $O(1)$ ")
```

Sample Output:

```
Enter the number of elements: 5
Enter the elements:
78
56
2
1
67

Sorted Array:
[1, 2, 56, 67, 78]

Execution Time: 0.00013534 seconds
```

Time Complexity:

Best Case : $O(n)$

Average Case : $O(n^2)$

Worst Case : $O(n^2)$

Space Complexity: $O(1)$

Selection Sort

Program:

```
import time

# Function for Selection Sort
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap the minimum element with the first unsorted element
        arr[i], arr[min_index] = arr[min_index], arr[i]

# User Input
n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Start timer
start_time = time.perf_counter()

# Perform Selection Sort
selection_sort(arr)

# End timer
end_time = time.perf_counter()

execution_time = end_time - start_time

# Display results
print("\nSorted Array:")
print(arr)

print(f"\nExecution Time: {execution_time:.8f} seconds")

print("\nTime Complexity:")
print("Best Case :  $O(n^2)$ ")
print("Average Case :  $O(n^2)$ ")
print("Worst Case :  $O(n^2)$ ")

print("\nSpace Complexity:  $O(1)$ ")
```

Sample Output:

```
Enter the number of elements: 5
Enter the elements:
90
3
87
4
6

Sorted Array:
[3, 4, 6, 87, 90]

Execution Time: 0.00016964 seconds

Time Complexity:
Best Case :  $O(n^2)$ 
Average Case :  $O(n^2)$ 
Worst Case :  $O(n^2)$ 

Space Complexity:  $O(1)$ 
```

Merge Sort

Program:

```
import time

# Merge Sort Function
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

# Merge Function
def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

# Main Program
n = int(input("Enter number of elements: "))
arr = list(map(int, input(f"Enter {n} elements: ").split()))

start = time.perf_counter()
sorted_arr = merge_sort(arr)
end = time.perf_counter()

execution_time = end - start

print("\nSorted Array:")
print(*sorted_arr)

print(f"\nExecution Time: {execution_time:.6f} seconds")

print("\nTime Complexity:")
print("Best Case : O(n log n)")
print("Average Case : O(n log n)")
print("Worst Case : O(n log n)")

print("\nSpace Complexity: O(n)")
```

Sample Output:

```
Enter number of elements: 5
Enter 5 elements: 6 78 2 21 69

Sorted Array:
2 6 21 69 78

Execution Time: 0.080129 seconds

Time Complexity:
Best Case : O(n log n)
Average Case : O(n log n)
Worst Case : O(n log n)

Space Complexity: O(n)
```

Quick Sort

Program:

```
import time

# Quick Sort Function
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]

    left = [x for x in arr[:-1] if x <= pivot]
    right = [x for x in arr[:-1] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)

# Main Program
n = int(input("Enter number of elements: "))
arr = list(map(int, input(f"Enter {n} elements: ").split()))

start = time.perf_counter()
sorted_arr = quick_sort(arr)
end = time.perf_counter()

execution_time = end - start

print("\nSorted Array:")
print(*sorted_arr)

print(f"\nExecution Time: {execution_time:.6f} seconds")

print("\nTime Complexity:")
print("Best Case :  $O(n \log n)$ ")
print("Average Case :  $O(n \log n)$ ")
print("Worst Case :  $O(n^2)$ ")

print("\nSpace Complexity:  $O(n)$ ")
```

Sample Output:

```
Enter number of elements: 5
Enter 5 elements: 65 78 3 23 6

Sorted Array:
3 6 23 65 78

Execution Time: 0.000132 seconds

Time Complexity:
Best Case :  $O(n \log n)$ 
Average Case :  $O(n \log n)$ 
Worst Case :  $O(n^2)$ 

Space Complexity:  $O(n)$ 
```

Insertion Sort

Program:

```
import time

# Insertion Sort Function
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr

# Main Program
n = int(input("Enter number of elements: "))
arr = list(map(int, input(f"Enter {n} elements: ").split()))

start = time.perf_counter()
sorted_arr = insertion_sort(arr)
end = time.perf_counter()

execution_time = end - start

print("\nSorted Array:")
print(*sorted_arr)

print(f"\nExecution Time: {execution_time:.6f} seconds")

print("\nTime Complexity:")
print("Best Case : O(n)")
print("Average Case : O(n^2)")
print("Worst Case : O(n^2)")

print("\nSpace Complexity: O(1)")
```

Sample Output:

```
Enter number of elements: 5
Enter 5 elements: 43 78 2 4 90

Sorted Array:
2 4 43 78 90

Execution Time: 0.000115 seconds

Time Complexity:
Best Case : O(n)
Average Case : O(n^2)
Worst Case : O(n^2)

Space Complexity: O(1)
```